

Assignment 2: Component building

Group 9: Benjamin Gallay, Robin Euger, Arantxa Meyre, Isabela Aterman

March 2026

1 Introduction

During the first assignment, we obtained a cleaned data set, ready to be used for economic analysis of Bangladeshi roads. To get a basic understanding of the data regarding simulated vehicle behavior, this assignment focuses on the driving behavior of trucks on the N1 national road. The simulation analyses trucks travelling along the road, evaluates travel times, examines delays caused by damaged bridges, and identifies bridges that may require repair. For this simulation, the code files provided in the assignment were used and modified to create a demonstration model that assesses the driving behavior of trucks on the N1 road.

In simulation modeling, it is important to properly abstract reality. This is typically done by decomposing the system into components that have high interaction internally, but low interaction between each other. As outlined in [1], several principles guide this process. For this assignment, it was the most relevant to use the "high cohesion, low coupling" principle. Accordingly, each component in our code is designed solely to perform one task and the components are passing to one another only data in the final form.

Taking these considerations into account, the goal of this assignment was modifying the components of the given code file to create a demo from which we can gauge the effect of bridge condition on travel times of trucks on the N1 Bangladeshi road.

2 Methods

2.1 Data reformatting

When analyzing the example model we noticed the data input format wasn't in alignment with the data files obtained in assignment one. In the `data_reformatting.py` file, data from `_roads3.csv` and `BMMS_overview.xlsx` is aggregated/modified and eventually deposited in the `roads.csv` with data properly formatted under the column names: `road`, `id`, `model_type`, `name`, `lat`, `lon`, `length`, `condition`, `lengthR`, `lengthL`, `conditionR`, `conditionL`, `bridgedual`

We choosed to not only use the bridge that where stated in the roads file via markers "BE/BS" but also include smaller bridges from BMMS file that were not in the roads file.

Key implementation details are:

- BMMS bridge condition classes **A/B/C/D** are mapped to numerical values **0/1/2/3**. For bridge segments where condition data is missing the condition is set to **0**.
- BMMS records are preprocessed per (**road**, **LRPName**): duplicate entries are aggregated using median values, left/right sides are detected from bridge names, and side-specific fields (**lengthL**, **lengthR**, **conditionL**, **conditionR**) are retained for dual bridges.
- Duplicate LRPs in roads and BMMS files are resolved : if the roads file row has a non-empty gap, the roads file row is kept; otherwise the BMMS file row is kept.
- Segment **model_type** is inferred from consecutive gap markers: **BS**→**BE** becomes **bridge**, **FS**→**FE** becomes **ferry**, and all others become **link**.
- Bridge length is taken from BMMS where available; if unavailable, the chainage difference is used as fallback.

- BMMS file bridge points are synthesized as start/end markers (BS/BE) using chainage and interpolated end coordinates to align BMMS bridges with the roads geometry.
- The beginning (source) and end (sink) of each road are explicitly added with `model_type = sourcesink`.
- Consecutive `link` segments are merged for efficiency, and final IDs are converted to numeric identifiers for model use.

With this approach we have all the needed data in one file making it much more easy to work with. The bridges that we simulate are the bridges stated in the roads file (with marker "BS" "BE") but also the bridges present in the BMMS file. Moreover this method works to generate the data for all the roads making it easier to generalize our model to a network of roads while the link merging allows us to have much less nodes and thus run the simulation faster.

road	id	model_type	name	lat	lon	length	condition	lengthR	lengthL	conditionR	conditionL	bridgedual
N1	1000018	bridge	N1.LRP011c.LRP011c.E	23.691	90.545	10.600	1					
N1	1000019	link	N1.LRP011c.E.LRP013a	23.691	90.546	841.400						
N1	1000020	bridge	N1.LRP013a.LRP013a.E	23.686	90.551	26.300	0	26.300	26.900	0	0	1
N1	1000021	link	N1.LRP013a.E.LRP013b	23.685	90.551	1.700						

Table 1: Sample rows from roads.csv

Then by simple truncation of the N1 to get the portion asked we have the road shape :

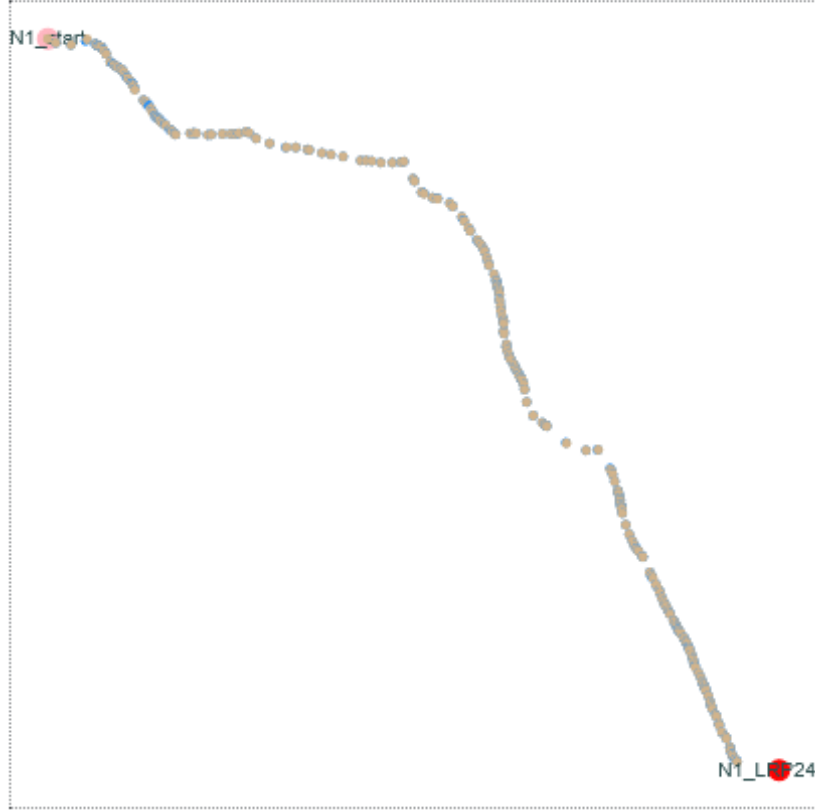


Figure 1: Shape of N1 road obtained via visualization python script

2.2 Modification of components

Only a handful of modifications to the simulation files needed to be made in order to account for the bridges being damaged :

- The `components.py` file is modified to take as input for the initialization of the class `BangladeshModel` the breakdown probabilities of all the conditions of bridges for the current scenario.
- The `get_delay_time` function from the `Bridge` class in `components.py`, which runs every time a truck tries to cross a bridge, is modified in order to take into account the randomness of the breakdown of bridges, and the possible time delays depending on the length of the bridge.

Note that for simplicity, the conditions of bridges are mapped from A, B, C, D to 0, 1, 2, 3 respectively.

2.3 Running the simulation and getting the results

For our demo, the code related to the simulation visualization was left mostly untouched. We focused on the `model_run.py` file instead to add a few features :

- A basic interface for the user to choose which scenario to run
- The matrix of breakdown probabilities as given for the assignment, ready to be sent to `model.py`
- Seed randomization using the current time (decimals in the nanoseconds)
- Output of simulation results

Furthermore, a `statistical_recorder.py` file was added in order to get the simulation data out of the model, write it to a file in the `/experiment/` folder, and return it to `model_run.py` to be printed in the console. We chose to write another file for this purpose so as not to clutter up the `components.py` file with code unrelated to components.

The `.csv` files written in the `/experiment/` contains, for each truck that have made it to the sink, their ID, and the time they took (in ticks (=minutes)). At the last row, the average time is written.

2.4 An analytical approach

We propose a second approach to this problem, which does not involve simulation at all. For this assignment, we would like to know the average time a truck takes to go from Chittagong to Dhaka, with bridges breaking down. We denote by $D_1, \dots, D_n$ the random variables equal to the delays of the n bridges of road N1. Then D the random variable of the total delay is equal to $D = \sum_{i=1}^n D_i$. Using the linearity of the expected value, we obtain $\mathbb{E} = \sum_{i=1}^n \mathbb{E}(D_i)$

The expected value of the total delay is the sum of the expected values of the delay of each bridge. The individual expected values are straightforward to compute, it is given by the product of the probability of the bridge breaking (depending on the scenario and the condition of the bridge) and the mean value of the distribution given in the assignment guidelines.

We add an `analytical_recorder.py` file to do this computation. It simply sums up the expected value for each bridge. We modify the `model.py` file to communicate with the recorder, and send the expected value for each bridge of the `.csv` dataset at the model's initialization. The recorder sends back the computed results to `model_run.py` to be displayed in the console.

This approach has many advantages regarding the scope of assignment 2 specifically. It is significantly faster, uses less resources, is more precise (not subject to the simulation's groundings to match exact ticks), and easier to implement. We do realize however, that the simulation approach probably lays the cornerstone for the simulation work we'll be doing in the next weeks.

2.5 Bonus questions

To answer the first bonus question, we added another feature to the `statistical_recorder.py`. It takes the form of a function `bridge_record`, called from the `components.py` file each time a bridge causes a delay for a truck. The function simply keeps count of how many ticks trucks have spend waiting at every bridge. At the end of scenarios 1-7, we rank the 5 bridges that have caused the most delays, and that should therefore be consolidated first.

This ranking is displayed in the console when the "bonus" option is chose in the console at the start of the program.

3 Results and Discussion

3.1 Scenarios outcomes

Scenarios	0	1	2	3	4	5	6	7	8
Average travel time (ticks)	291	291	291	322	355	421	550	726	1165

Table 2: Average travel time for all scenarios, 10 seeds

In scenarios 0, 1 and 2, all the trucks take exactly 291 ticks to travel from Chittagong to Dhaka. This behavior is coherent since there are no bridges of condition D in the portion of the N1 road between Chittagong

and Dhaka. Since in these scenarios the only bridges susceptible to break down are bridges of condition D, this behavior is expected.

According to google maps, the distance between the source and sink LRPs chosen in Chittagong and Dhaka along the N1 road is 239 km. This results in a theoretical 286,8 ticks to travel. We believe the result of our simulation is therefore coherent when inaccuracy at the dataset level is taken into account.

We observe increases in the travel time for the next scenarios. This is expected, as the probability of bridges breaking down increase. This increase is significant as well, with travel times four times as long in scenario 8 compared to the scenario 0 baseline.

3.2 Seeds

To evaluate if only 10 replications is enough for our simulation, we did 10 consecutive runs of scenario 8 (which should be the most inconsistent of all of them) and here are the results :

Instance	1	2	3	4	5	6	7	8	9	10
Average travel time (ticks)	1165	1166	1166	1169	1167	1165	1164	1166	1167	1164

Table 3: Average travel time scenario 8, 10 runs, 10 replications each

We only observe a difference of 5 ticks (0.4%) between the biggest and the smallest value, which seems a good enough precision for our analysis. Therefore, we claim that 10 replications are sufficient for this work.

To choose the seeds to run the simulation on, we take a few digits in the order of the microsecond from the current time. The consequent runs are done using the consecutive seeds in the order of natural numbers.

3.3 Bonus questions

3.3.1 Bridges to be consolidated first

When choosing the "bonus" option at the start of the program, the ranking of the bridges that have caused the most delays is displayed. Note that this ranking can vary (mostly those in position 3, 4 and 5) since many bridges have very similar scores (which is the total delay time). However, we consistently find that the bridges at the top of this ranking are among the longest bridges of condition B or C (since no bridge of condition D is in the studied portion of the N1 road).

Two bridges however are consistently in position 1 and 2, bridges 1000030 and 1000006, with scores of approximately 583000 and 433000 ticks respectively (depending on the simulation runs), well above the rest of the bridges. These are the bridges that would need to be consolidated first. However, our criterion only considers the total time delay caused by a bridge, not the cost of its repair.

3.3.2 Modularity

Several modifications towards modularity have been done, mostly for the sake of clarity. The code for extracting simulation data out of the model has been move to dedicated files `statistical_recorder.py` and `analytical_recorder.py`, as not to clutter up the `model.py` and `components.py` files with code unrelated to the models. The different functions of the files have been implemented in functions as well.

Regarding the dataset reformatting code, it has been divided in a pre-formatting code and a main logic code. The first code consist of reformatting the BMMS data to roads3-like data. And inside each of this files the task are well separated inside small function. With such modularity the code is really clear and debugging was an easy task.

Furthermore a README file is given in each folder to explain in detail the function of the code in the folder specifically.

4 Limitations and potential improvements

As this is a demo, the scenarios used to study the driving times are very basic. Namely, the probability of a bridge breaking down and the accompanying repair time, based on bridge length. These are inadequate metrics to accurately test delay times caused by bridge conditions. Type of damage, repair costs, accessibility of the bridge are parameters that can affect the delay time caused by the bridge conditions.

Beyond the repair of bridges, traffic also plays a big role in the driving behavior of vehicles. This demo model

only illustrates the driving behavior of truck, on a singular road, in one direction. Realistically the road is occupied with other vehicles, whose driving behavior affects each other.

We also lack much information to give a proper answer for the bonus question : identifying the bridges that should be repaired first. In order to answer this question properly, we would need more information about the cost of repairing a bridge given its condition and its length, and we would need to simulate alternative paths that can go around a broken bridge.

Consequently, we found a few ways the simulation could be improved to give a better answer to the stakeholders :

- Building a simulation of several roads at the same time, to simulate rerouting in the case of a broken bridge
- Introducing correlation of the breaking of different bridges close to each other to simulate localized floods or mudslides
- Using bridge building data to take the cost of repair in the criterion to choose which bridges need to be consolidated

5 Acknowledgment

5.1 Team members main roles

- Robin: Data reformatting
- Benjamin: Modification of simulation files
- Arantxa: writing
- Isabela: writing

5.2 AI statement

Grammarly was used as an aid during the writing process.

No AI was used in writing the code in relation to the simulation

LLMs were used to write parts of the data reformatting algorithm, more precisely on translating the by row code to vectorization to improve run time

References

- [1] Marko A Hofmann. Criteria for decomposing systems into components in modeling and simulation: Lessons learned with military simulations. *Simulation*, 80(7-8):357–365, 2004.